

> myShell <

Laborbericht

- **Projekt:** Entwicklung einer eigenen Linux-Shell in C (myShell)
- **Autor:** Adrian Pinggera
- **Klasse:** 3IB
- **Schule:** Technologische Fachoberschule "Max Valier" Bozen
- **Schuljahr:** 2025/26
- **Auftragsdatum:** 30.03.2026
- **Abgabedatum:** 30.04.2026
- **Art des Dokumentes:** Laborbericht

Inhaltsverzeichnis

1. Arbeitsauftrag
2. Theoretischer Hintergrund
3. Analyse & Umsetzung 3.1 Lasten- und Pflichtenheft
4. Planung und Programmablauf

1. Arbeitsauftrag

"Entwickle eine kleine Shell, die Programmaufrufe des Benutzers einlesen, parsen und ausführen kann. Nach der Durchführung kann der Benutzer weiteren Aufrufe durchführen. Zu berücksichtigen sind übergebene Argumente und evtl. die Angabe eines Pfades für den Aufruf. Neben dem Programm wird ein Laborbericht, ein Handbuch und der Testbericht abgegeben. Jeder gibt seine eigene Arbeit ab." - Moodle

2. Theoretischer Hintergrund

Eine Shell fungiert als Schnittstelle zwischen dem Benutzer und dem Betriebssystem. Sie nimmt Textbefehle entgegen und leitet diese an den Kernel weiter. Um dies in der Programmiersprache C umzusetzen, habe ich mich mit den Systemaufrufen `fork()` (zum Erzeugen eines parallelen Kindprozesses), `execvp()` (zum Ausführen eines Programms mit Argumenten und Pfad-Berücksichtigung) und `wait()` (damit die Shell wartet, bis der Befehl fertig ist) beschäftigt. Zusätzlich habe ich mich in die `readline`-Bibliothek eingearbeitet, um Pfeiltasten-Navigation (History) zu ermöglichen, und das Tool `fastfetch` integriert, um der Shell beim Start eine individuelle Optik zu geben.

3. Analyse & Umsetzung

Lasten- und Pflichtenheft

Aus dem Arbeitsauftrag habe ich folgende Kernanforderungen abgeleitet und umgesetzt:

- **Einlesen und Parsen:** Die Eingaben werden per `readline` eingelesen und mit `strtok` an den Leerzeichen aufgespalten (geparst), um den eigentlichen Befehl von den übergebenen Argumenten zu trennen.
- **Pfad und Ausführung:** Durch die Nutzung von `execvp` wird automatisch in den System-Umgebungsvariablen (`PATH`) nach dem Programm gesucht. Die Eingabe von absoluten oder relativen Pfaden (z.B. `./programm`) wird direkt unterstützt.

- **Endlosschleife:** Eine while(1)-Schleife sorgt dafür, dass nach Beendigung eines Befehls sofort ein neuer Prompt für weitere Eingaben erscheint.
- **Zusatz-Features:** Um das Projekt abzurunden, habe ich eine farbige Prompt-Anzeige (inklusive aktuellem Verzeichnis), einen "Fastfetch"-Startbildschirm mit eigenem ASCII-Logo sowie ein Makefile zur systemweiten Installation und Deinstallation implementiert.

Implementierung & Versionsverwaltung

Für die Entwicklung dieses Projekts wurde Git als Versionsverwaltungssystem eingesetzt. Der gesamte Quellcode, die Konfigurationsdateien sowie die Dokumentation wurden in einem GitHub-Repository verwaltet. Dies ermöglichte nicht nur eine strukturierte Historie der Entwicklungsschritte (Commits), sondern stellt auch die Integrität des Projekts für externe Nutzer sicher.

- Repository-Link: `git@github.com:muryo-void/myShell.git`
- Deployment-Strategie: Über das Makefile wurde sichergestellt, dass das Projekt direkt nach dem Klonen vom Repository einsatzbereit ist.

4. Planung und Programmablauf

Um eine saubere Trennung zwischen Quellcode, ausführbaren Dateien und Konfigurationen zu gewährleisten, wurde folgende Struktur geplant und umgesetzt:

```
muryo@AdriansLaptop:~/Schule/3_Klasse_2025_26/TP/myShell$ tree
.
├── README.md
├── config
│   ├── ascii.txt
│   └── design.jsonc
├── docs
│   ├── Handbuch.odt
│   ├── Laborbericht.odt
│   └── Testbericht.odt
├── makefile
├── myShell
├── src
│   └── myShell.c
```

Der logische Ablauf meiner main-Funktion sieht wie folgt aus:

1. Beim Start wird fastfetch aufgerufen, um das Logo und die Systemdaten zu laden.
2. Der aktuelle Benutzer (`getenv("USER")`) und das Arbeitsverzeichnis (`getcwd`) werden für den farbigen Prompt ermittelt.
3. Die Endlosschleife beginnt: Warte auf die Benutzereingabe.

4. Die Eingabe wird in die History aufgenommen und in ihre Einzelteile (Token) zerlegt.
5. Built-in Befehle (wie `cd` für Verzeichniswechsel oder `exit`) werden direkt in der Shell verarbeitet, da sie keinen eigenen Prozess benötigen.
6. Für alle anderen Befehle wird der Prozess mittels `fork()` gespalten. Der Kindprozess führt das Programm aus, der Elternprozess wartet.